

Project: Modello di Vicsek per branchi animali

moto collettivo, auto-propulsione e transizioni ordine–disordine

1. Obiettivi della dispensa

Questa dispensa introduce il modello di Vicsek come caso di studio per un corso di metodi computazionali per modelli stocastici. Il problema centrale e' capire come un insieme di agenti auto-propellenti, dotati soltanto di interazioni locali e di una piccola componente rumorosa, possa generare moto collettivo ordinato a scala macroscopica.

Gli obiettivi sono sei:

1. formalizzare il modello microscopico di particelle auto-propellenti nel piano;
2. distinguere chiaramente tra stato del sistema, regole di aggiornamento e parametri di controllo;
3. introdurre osservabili quantitative per misurare ordine collettivo, clustering e correlazioni spaziali;
4. mostrare come il rumore e la densita' influenzino la transizione tra regime disordinato e regime ordinato;
5. usare il modello come base per simulazioni numeriche, analisi parametrica e visualizzazione di traiettorie;
6. discutere il modello come esempio paradigmatico di emergenza di ordine in sistemi lontani dall'equilibrio.

Dal punto di vista didattico, questo modulo e' particolarmente forte perche' collega fisica statistica, biologia del comportamento collettivo, simulazione agent-based e analisi di transizioni collettive.

2. Motivazione generale

Molti sistemi biologici mostrano forme di coordinamento collettivo sorprendenti:

- stormi di uccelli;
- branchi di pesci;
- sciami di insetti;
- gruppi di cellule mobili;
- colonie batteriche in movimento;

- sistemi robotici ispirati al comportamento animale.

In questi sistemi non esiste necessariamente un controllore centrale. L'ordine macroscopico emerge dall'interazione locale tra molti individui che seguono regole semplici. Questo e' precisamente il tipo di problema che rende il modello di Vicsek un eccellente case study per il corso.

Il punto concettuale fondamentale e' il seguente: una popolazione di agenti, inizialmente disordinata, puo' sviluppare spontaneamente una direzione di moto coerente anche in presenza di rumore. Il problema non e' soltanto descrivere il moto dei singoli, ma capire come si produca una proprieta' collettiva emergente.

3. Definizione formale del modello

3.1 Particelle e dominio spaziale

Consideriamo N particelle puntiformi che si muovono in un dominio bidimensionale quadrato di lato L .

Per evitare effetti di bordo, assumiamo in prima istanza condizioni periodiche al contorno. In altre parole, il dominio e' topologicamente equivalente a un toro bidimensionale.

La densita' media del sistema e' quindi

$$\rho = \frac{N}{L^2}.$$

3.2 Variabili di stato

Ogni particella i e' caratterizzata da:

- una posizione

$$r_i(t) = (x_i(t), y_i(t)) \in [0, L]^2;$$

- un angolo di direzione

$$\theta_i(t) \in [0, 2\pi);$$

- una velocita' di modulo costante v_0 .

Il vettore velocita' della particella e' quindi

$$v_i(t) = v_0(\cos \theta_i(t), \sin \theta_i(t)).$$

Lo stato completo del sistema al tempo t e' dato dalla collezione

$$X_t = (r_1(t), \theta_1(t), \dots, r_N(t), \theta_N(t)).$$

3.3 Vicinato locale

Per ogni particella i , definiamo l'insieme dei vicini come

$$\mathcal{N}_i(t) = \{j : \|r_j(t) - r_i(t)\|_{\text{per}} \leq R\},$$

dove $\|\cdot\|_{\text{per}}$ indica la distanza euclidea minima tenendo conto delle condizioni periodiche, e R e' il raggio di interazione.

In molte implementazioni si include anche la particella i nel proprio vicinato. Questa scelta e' conveniente e non altera qualitativamente il modello.

4. Dinamica microscopica

4.1 Aggiornamento della direzione

Il cuore del modello e' la tendenza all'allineamento locale. A ogni passo temporale di ampiezza Δt , la particella i aggiorna la propria direzione verso la direzione media dei vicini, con aggiunta di rumore:

$$\theta_i(t + \Delta t) = \text{Arg} \left(\sum_{j \in \mathcal{N}_i(t)} e^{i\theta_j(t)} \right) + \eta \xi_i(t),$$

dove:

- $\text{Arg}(z)$ restituisce l'argomento del numero complesso z ;
- $\eta \geq 0$ e' l'ampiezza del rumore;
- $\xi_i(t)$ e' una variabile casuale uniforme, per esempio in $[-1/2, 1/2]$ oppure in $[-\pi, \pi]$ a seconda della convenzione adottata.

Per coerenza, conviene fissare una convenzione esplicita. Una scelta semplice e':

$$\xi_i(t) \sim U[-1/2, 1/2],$$

cosi' che il rumore effettivo sia distribuito uniformemente nell'intervallo $[-\eta/2, \eta/2]$.

4.2 Aggiornamento della posizione

Una volta aggiornata la direzione, la posizione evolve secondo

$$r_i(t + \Delta t) = r_i(t) + v_0(\cos \theta_i(t + \Delta t), \sin \theta_i(t + \Delta t))\Delta t.$$

Dopo l'aggiornamento, si reimposta la posizione nel dominio periodico tramite riduzione modulo L .

4.3 Natura stocastica del modello

La successione $\{X_t\}_{t \geq 0}$ definisce un processo stocastico a tempo discreto. La componente casuale entra direttamente nella dinamica, attraverso il rumore angolare che perturba l'allineamento locale.

Questo punto e' centrale per il corso: l'ordine collettivo non emerge nonostante il rumore, ma in equilibrio dinamico con esso. Il comportamento macroscopico del sistema dipende dall'interazione tra:

- densita' di particelle;
- velocita' propria;
- raggio di interazione;
- intensita' del rumore.

5. Interpretazione dei parametri

I parametri fondamentali del modello sono i seguenti.

5.1 Numero di particelle N

Misura la dimensione del sistema. In simulazione, valori piu' grandi riducono gli effetti di popolazione finita ma aumentano il costo computazionale.

5.2 Lato del dominio L

Controlla la dimensione spaziale disponibile. A parita' di N , variazioni di L modificano la densita' ρ .

5.3 Densita' ρ

La densita' media

$$\rho = \frac{N}{L^2}$$

e' una quantita' cruciale. A densita' troppo bassa, le interazioni locali diventano rare e l'ordine collettivo e' difficile da sostenere.

5.4 Raggio di interazione R

Determina la scala locale dell'allineamento. Valori piu' grandi implicano vicinati mediamente piu' numerosi.

5.5 Velocita' propria v_0

Controlla la rapidita' con cui le particelle si spostano nello spazio. Essa determina, insieme a Δt , la lunghezza del passo spaziale

$$\ell = v_0 \Delta t.$$

5.6 Rumore η

E' il parametro piu' importante per la fenomenologia ordine-disordine. Per η piccolo, l'allineamento domina; per η grande, il sistema tende al disordine.

5.7 Passo temporale Δt

In molte implementazioni si pone $\Delta t = 1$ per semplicita'. Tuttavia conviene mantenerlo esplicito in fase di definizione, per chiarire il ruolo della discretizzazione.

6. Osservabili da misurare

Per analizzare la dinamica servono osservabili quantitative.

6.1 Parametro d'ordine globale

La misura piu' importante e' il parametro d'ordine di polarizzazione

$$\Phi(t) = \frac{1}{N v_0} \left| \sum_{i=1}^N v_i(t) \right|.$$

Equivalentemente,

$$\Phi(t) = \frac{1}{N} \left| \sum_{i=1}^N (\cos \theta_i(t), \sin \theta_i(t)) \right|.$$

Interpretazione:

- $\Phi(t) \approx 1$ indica forte allineamento globale;
- $\Phi(t) \approx 0$ indica direzioni distribuite in modo quasi isotropo.

6.2 Parametro d'ordine medio stazionario

Per ridurre le fluttuazioni, si puo' misurare la media temporale su una finestra finale:

$$\overline{\Phi} = \frac{1}{T_2 - T_1 + 1} \sum_{t=T_1}^{T_2} \Phi(t).$$

Questa osservabile e' utile per costruire diagrammi di fase empirici, ad esempio $\overline{\Phi}$ in funzione di η o di ρ .

6.3 Fluttuazioni del parametro d'ordine

Una misura della suscettivita' empirica e'

$$\chi_{\Phi} = N \left(\langle \Phi^2 \rangle - \langle \Phi \rangle^2 \right).$$

Picchi di χ_{Φ} possono segnalare una regione di transizione tra disordine e ordine.

6.4 Correlazione direzionale

Una possibile funzione di correlazione spaziale e'

$$C(r) = \frac{\left\langle \sum_{i \neq j} \delta_r(\|r_i - r_j\|) \hat{v}_i \cdot \hat{v}_j \right\rangle}{\left\langle \sum_{i \neq j} \delta_r(\|r_i - r_j\|) \right\rangle},$$

dove $\hat{v}_i = v_i/v_0$ e δ_r indica una binning function su distanze simili a r .

Questa misura quantifica quanto l'allineamento persista a distanza crescente.

6.5 Cluster spaziali

Si possono definire cluster come componenti connesse nel grafo geometrico istantaneo: due particelle sono collegate se la loro distanza e' minore o uguale a una soglia, tipicamente R .

Osservabili naturali sono:

- dimensione del cluster piu' grande;
- numero totale di cluster;
- distribuzione delle dimensioni dei cluster.

6.6 Tempo di formazione dell'ordine

Dato un valore soglia Φ^* , si puo' definire il tempo di formazione dell'ordine come

$$\tau_{\text{ord}} = \inf\{t : \Phi(t) \geq \Phi^*\}.$$

Questa osservabile e' utile per confrontare la rapidita' di organizzazione in regimi diversi.

7. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande precise.

1. Come dipende il livello di ordine collettivo dal rumore η ?
2. Esiste una soglia o regione critica che separa moto disordinato e moto ordinato?
3. A densita' maggiore il sistema si organizza piu' facilmente?
4. Le strutture spaziali emergenti sono omogenee oppure compaiono bande e cluster mobili?
5. Come cambia il comportamento al variare di R , v_0 e della taglia del sistema?
6. Quanto sono robusti i risultati rispetto alla scelta di rumore, alle condizioni iniziali e alle condizioni al contorno?

8. Pseudocodice del modello

Di seguito una versione semplice con aggiornamento sincrono.

8.1 Input

- numero di particelle N
- lato del dominio L
- raggio di interazione R
- velocita' v_0
- rumore η
- passo temporale Δt
- numero massimo di passi T

8.2 Pseudocodice generale

1. Inizializza casualmente le posizioni:

$$r_i(0) \sim U([0, L]^2).$$

2. Inizializza casualmente le direzioni:

$$\theta_i(0) \sim U([0, 2\pi)).$$

3. Per ogni tempo $t = 0, \dots, T - 1$:
 - per ogni particella i , costruisci il vicinato $\mathcal{N}_i(t)$;
 - calcola la direzione media locale;
 - estrai un rumore casuale $\xi_i(t)$;
 - aggiorna la direzione $\theta_i(t + \Delta t)$;
 - aggiorna la posizione $r_i(t + \Delta t)$;

- applica le condizioni periodiche;
- misura le osservabili:

$$\Phi(t), \quad C(r), \quad \text{cluster}, \quad \tau_{\text{ord}} \text{ se necessario.}$$

4. Salva le traiettorie delle osservabili.
5. Ripeti per molte realizzazioni indipendenti e calcola medie e fluttuazioni.

8.3 Nota computazionale

La parte piu' costosa del codice e' tipicamente la ricerca dei vicini. Una implementazione elementare usa il confronto di tutte le coppie, con costo dell'ordine di N^2 per passo. In una versione piu' avanzata si possono usare celle spaziali per ridurre il costo.

9. Commento didattico sul pseudocodice

Dal punto di vista computazionale, il modello permette di esercitare competenze fondamentali:

- rappresentazione di particelle in spazio continuo;
- aggiornamento stocastico di variabili angolari;
- uso di condizioni periodiche;
- definizione di osservabili macroscopiche a partire da regole microscopiche;
- confronto tra traiettorie singole e medie di ensemble;
- analisi parametrica e visualizzazione di una transizione collettiva.

Il modello e' inoltre abbastanza semplice da essere implementato in Python, Julia, Matlab o C senza difficolta' concettuali eccessive.

10. Schema del laboratorio

10.1 Laboratorio 1 - Implementazione del modello base

Obiettivo

Implementare il modello di Vicsek con aggiornamento sincrono e visualizzare l'emergenza del moto collettivo.

Attivita'

1. fissare valori iniziali, ad esempio

$$N = 200, \quad L = 20, \quad R = 1, \quad v_0 = 0.03, \quad \Delta t = 1;$$

2. inizializzare posizioni e direzioni casuali;
3. simulare una traiettoria temporale per diversi valori di η ;
4. rappresentare le particelle nel piano con vettori di direzione;
5. calcolare e tracciare $\Phi(t)$.

Domande guida

- il sistema sviluppa una direzione collettiva?
- il valore di $\Phi(t)$ cresce rapidamente oppure lentamente?
- il rumore distrugge completamente l'ordine o ne resta una traccia?

Output richiesto

- codice sorgente;
- grafici temporali di $\Phi(t)$;
- snapshot spaziali del sistema;
- breve commento interpretativo.

10.2 Laboratorio 2 - Transizione ordine–disordine

Obiettivo

Studiare la dipendenza del parametro d'ordine dal rumore.

Attività

1. fissare N , L , R e v_0 ;
2. scegliere una griglia di valori di η ;
3. per ogni valore, eseguire più simulazioni indipendenti;
4. calcolare $\bar{\Phi}$ e, opzionalmente, χ_Φ ;
5. costruire il grafico di $\bar{\Phi}$ in funzione di η .

Domande guida

- esiste una regione di passaggio netto tra ordine e disordine?
- dove sono massime le fluttuazioni di Φ ?
- i risultati dipendono molto dalla taglia del sistema?

Output richiesto

- tabella o grafico di $\bar{\Phi}(\eta)$;
- eventualmente grafico di $\chi_\Phi(\eta)$;
- commento sulla presenza di una transizione collettiva.

10.3 Laboratorio 3 - Ruolo della densità

Obiettivo

Studiare come la densità influenzi la capacità del sistema di organizzarsi.

Attività

1. mantenere fisso N e variare L ;
2. oppure mantenere fisso L e variare N ;

3. per ogni densita', simulare il sistema a rumore fissato;
4. confrontare $\bar{\Phi}$, numero di cluster e tempo di ordinamento.

Domande guida

- esiste una densita' minima sotto la quale il flocking non si sostiene?
- la maggiore densita' accelera l'ordine?
- i cluster diventano piu' compatti oppure piu' estesi?

Output richiesto

- grafici di $\bar{\Phi}$ in funzione di ρ ;
- statistiche sui cluster;
- commento comparativo.

10.4 Laboratorio 4 - Correlazioni e strutture spaziali

Obiettivo

Studiare non solo l'ordine globale, ma anche la struttura spaziale del moto collettivo.

Attivita'

1. calcolare una funzione di correlazione direzionale;
2. definire cluster geometrici tramite distanza;
3. misurare la dimensione del cluster piu' grande;
4. osservare la presenza di configurazioni spazialmente organizzate.

Domande guida

- l'ordine globale coincide sempre con cluster spaziali compatti?
- la correlazione direzionale decade rapidamente oppure lentamente?
- il sistema mostra bande, fronti o altre strutture coerenti?

Output richiesto

- grafici di correlazione;
- statistiche di cluster;
- discussione qualitativa delle configurazioni osservate.

11. Una possibile estensione teorica

Per una lettura piu' avanzata, si puo' collegare il modello microscopico a descrizioni cinetiche o idrodinamiche. In questa prospettiva, il sistema non viene descritto seguendo ogni singola particella, ma tramite campi continui di densita' e velocita' media.

Senza entrare nei dettagli tecnici, il punto importante e' che il modello di Vicsek puo' essere letto come ponte tra:

- simulazione agent-based;
- equazioni cinetiche per distribuzioni in spazio e direzione;
- descrizioni collettive di tipo campo medio o idrodinamico.

Questo rende il modulo particolarmente interessante per studenti con sensibilita' piu' fisica o matematica.

12. Possibili estensioni del modello

Il progetto base puo' essere ampliato in molte direzioni.

12.1 Interazioni topologiche

Invece di considerare tutti i vicini entro distanza R , ogni particella puo' allinearsi con i suoi k vicini piu' prossimi. Questa variante e' spesso rilevante in applicazioni etologiche.

12.2 Repulsione e attrazione

Si possono aggiungere termini di repulsione a corto raggio e attrazione a medio raggio, ottenendo modelli piu' ricchi di schooling e flocking.

12.3 Ostacoli o eterogeneita' ambientale

Il dominio puo' contenere regioni proibite, ostacoli o gradienti esterni che modificano il moto.

12.4 Predatori o leader

Una o poche particelle possono avere un comportamento speciale, per esempio guidare il gruppo oppure disturbare l'ordine.

12.5 Velocita' variabile

Invece di fissare v_0 , si puo' introdurre una dinamica della velocita', utile per studiare frenata, accelerazione o eterogeneita' individuale.

12.6 Rumore non uniforme

Il rumore puo' dipendere localmente dalla densita' o dalla configurazione del gruppo, invece di essere identico per tutte le particelle.

13. Perché questo è un ottimo case study per il corso

Questo progetto è particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, mostra in modo molto pulito come una regola microscopica locale produca una proprietà macroscopica emergente.

Secondo, introduce una forma di stocasticità direttamente inserita nella dinamica, non soltanto nelle condizioni iniziali.

Terzo, consente di lavorare con osservabili molto chiare e visivamente intuitive, come il parametro d'ordine e i cluster.

Quarto, collega naturalmente discipline diverse:

- fisica statistica;
- etologia quantitativa;
- sistemi complessi;
- robotica collettiva;
- simulazione numerica.

Quinto, è un modello estremamente flessibile: da una versione minima si può passare a versioni più sofisticate senza perdere chiarezza concettuale.

14. Conclusione

Il modello di Vicsek è uno dei casi di studio più efficaci per mostrare come l'ordine collettivo possa emergere da interazioni locali rumorose tra agenti auto-propellenti.

Dal punto di vista metodologico, il progetto combina in modo molto naturale:

- modellizzazione microscopica;
- simulazione stocastica;
- analisi di osservabili macroscopiche;
- studio di transizioni collettive;
- interpretazione interdisciplinare.

Il punto concettuale più importante è che il moto coordinato non viene imposto dall'esterno, ma nasce spontaneamente dall'interazione tra molte unità elementari.

15. Bibliografia minima

1. Vicsek, T., Czir'ok, A., Ben-Jacob, E., Cohen, I., and Shochet, O. Novel type of phase transition in a system of self-driven particles.

2. Toner, J., and Tu, Y. Flocks, herds, and schools: A quantitative theory of flocking.
 3. Cavagna, A., and Giardina, I. Bird flocks as condensed matter.
 4. Sumpter, D. J. T. Collective Animal Behavior.
 5. Romanczuk, P., B"ar, M., Ebeling, W., Lindner, B., and Schimansky-Geier, L. Active Brownian particles.
-

Appendice A – Pseudocodice Python quasi eseguibile

Questa appendice propone una implementazione elementare del modello di Vicsek in due dimensioni, con:

- condizioni periodiche al contorno;
- aggiornamento sincrono delle direzioni;
- rumore angolare uniforme;
- misura del parametro d'ordine globale;
- una semplice procedura per identificare cluster geometrici;
- una routine per esplorare la dipendenza dal rumore.

L'obiettivo non e' fornire una libreria ottimizzata, ma una base chiara e didatticamente trasparente.

A.1 Import e convenzioni

```
import math
import random
from collections import deque
```

Assumiamo le seguenti convenzioni:

- il sistema vive in un quadrato di lato L ;
- le posizioni sono coppie (x, y) con $0 \leq x < L$ e $0 \leq y < L$;
- ogni particella ha velocità di modulo costante v_0 ;
- la direzione di moto e' rappresentata da un angolo θ in radianti;
- il rumore e' uniforme in $[-\eta/2, \eta/2]$.

A.2 Utility di base

```
def make_rng(seed=None):
    """Restituisce un generatore pseudo-casuale locale."""
    return random.Random(seed)

def wrap_position(x, L):
```

```

    """
    Riporta una coordinata  $x$  nel dominio periodico  $[0, L)$ .
    """
    return x % L

def wrap_angle(theta):
    """
    Riporta un angolo nell'intervallo  $[0, 2\pi)$ .
    """
    two_pi = 2.0 * math.pi
    return theta % two_pi

```

A.3 Distanza periodica e spostamento minimo

Per costruire il vicinato locale occorre calcolare la distanza minima tra due particelle tenendo conto delle condizioni periodiche.

```

def periodic_displacement(x1, y1, x2, y2, L):
    """
    Restituisce lo spostamento minimo periodico dal punto 1 al punto 2.
    """
    dx = x2 - x1
    dy = y2 - y1

    if dx > 0.5 * L:
        dx -= L
    elif dx < -0.5 * L:
        dx += L

    if dy > 0.5 * L:
        dy -= L
    elif dy < -0.5 * L:
        dy += L

    return dx, dy

def periodic_distance(x1, y1, x2, y2, L):
    """
    Distanza euclidea minima periodica tra due punti del dominio.
    """
    dx, dy = periodic_displacement(x1, y1, x2, y2, L)
    return math.sqrt(dx * dx + dy * dy)

```

A.4 Inizializzazione del sistema

```
def initialise_vicsek_state(N, L, rng=None):
    """
    Inizializza posizioni e direzioni in modo casuale.

    Restituisce:
    - positions: lista di coppie (x, y)
    - angles: lista di angoli theta
    """
    if rng is None:
        rng = make_rng()

    positions = []
    angles = []

    for _ in range(N):
        x = rng.random() * L
        y = rng.random() * L
        theta = rng.random() * 2.0 * math.pi

        positions.append((x, y))
        angles.append(theta)

    return positions, angles
```

A.5 Costruzione del vicinato

Questa implementazione usa il metodo piu' semplice: per ogni particella confronta la distanza con tutte le altre. Il costo computazionale per passo e' dell'ordine di N^2 .

```
def neighbours_of_particle(i, positions, R, L):
    """
    Restituisce la lista degli indici j tali che la particella j
    sia entro distanza R dalla particella i.

    Si include anche la particella i stessa nel proprio vicinato.
    """
    xi, yi = positions[i]
    neigh = []

    for j, (xj, yj) in enumerate(positions):
        d = periodic_distance(xi, yi, xj, yj, L)
        if d <= R:
            neigh.append(j)
```

```
return neigh
```

A.6 Aggiornamento sincrono delle direzioni

Il modello di Vicsek aggiorna prima tutte le direzioni, e solo dopo tutte le posizioni.

```
def local_average_angle(i, positions, angles, R, L):
    """
    Calcola la direzione media locale della particella i
    usando la rappresentazione complessa / vettoriale.
    """
    neigh = neighbours_of_particle(i, positions, R, L)

    sx = 0.0
    sy = 0.0

    for j in neigh:
        sx += math.cos(angles[j])
        sy += math.sin(angles[j])

    return math.atan2(sy, sx)


def vicsek_step(positions, angles, L, R, v0, eta, dt=1.0, rng=None):
    """
    Esegue un singolo passo del modello di Vicsek con aggiornamento sincrono.

    Parametri:
    - positions: lista di posizioni
    - angles: lista di angoli
    - L: lato del dominio
    - R: raggio di interazione
    - v0: modulo della velocita'
    - eta: ampiezza del rumore angolare
    - dt: passo temporale

    Restituisce:
    - new_positions
    - new_angles
    """
    if rng is None:
        rng = make_rng()

    N = len(positions)
```



```

# 1. Aggiornamento sincrono delle direzioni
new_angles = [0.0 for _ in range(N)]

for i in range(N):
    theta_mean = local_average_angle(i, positions, angles, R, L)
    noise = rng.uniform(-0.5 * eta, 0.5 * eta)
    new_angles[i] = wrap_angle(theta_mean + noise)

# 2. Aggiornamento delle posizioni
new_positions = []

for i in range(N):
    x, y = positions[i]
    theta = new_angles[i]

    x_new = x + v0 * math.cos(theta) * dt
    y_new = y + v0 * math.sin(theta) * dt

    x_new = wrap_position(x_new, L)
    y_new = wrap_position(y_new, L)

    new_positions.append((x_new, y_new))

return new_positions, new_angles

```

A.7 Parametro d'ordine globale

```

def order_parameter(angles):
    """
    Calcola il parametro d'ordine globale  $\Phi(t)$ .

     $\Phi = (1/N) * |\sum_i (\cos \theta_{i,t}, \sin \theta_{i,t})|$ 
    """
    N = len(angles)

    sx = 0.0
    sy = 0.0

    for theta in angles:
        sx += math.cos(theta)
        sy += math.sin(theta)

    return math.sqrt(sx * sx + sy * sy) / N

```

A.8 Funzioni di supporto per le traiettorie

```
def velocity_vectors(angles, v0):  
    """  
    Restituisce la lista dei vettori velocita'.  
    """  
    velocities = []  
    for theta in angles:  
        vx = v0 * math.cos(theta)  
        vy = v0 * math.sin(theta)  
        velocities.append((vx, vy))  
    return velocities  
  
def mean_direction(angles):  
    """  
    Direzione media globale, utile per analisi qualitative.  
    """  
    sx = 0.0  
    sy = 0.0  
  
    for theta in angles:  
        sx += math.cos(theta)  
        sy += math.sin(theta)  
  
    return math.atan2(sy, sx)
```

A.9 Cluster geometrici

Per una prima analisi, definiamo un cluster come una componente connessa del grafo geometrico istantaneo: due particelle sono adiacenti se la loro distanza e' minore o uguale a `R_cluster`.

Per semplicita', qui usiamo `R_cluster = R`, ma si potrebbe scegliere una soglia diversa.

```
def adjacency_list_geometric(positions, L, R_cluster):  
    """  
    Costruisce la lista di adiacenza del grafo geometrico istantaneo.  
    """  
    N = len(positions)  
    adj = [[] for _ in range(N)]  
  
    for i in range(N):  
        xi, yi = positions[i]  
        for j in range(i + 1, N):  
            xj, yj = positions[j]
```

```

        d = periodic_distance(xi, yi, xj, yj, L)
        if d <= R_cluster:
            adj[i].append(j)
            adj[j].append(i)

    return adj

def connected_components(adj):
    """
    Restituisce la lista delle componenti connesse,
    ciascuna come lista di indici di particelle.
    """
    N = len(adj)
    visited = [False] * N
    components = []

    for start in range(N):
        if visited[start]:
            continue

        queue = deque([start])
        visited[start] = True
        comp = []

        while queue:
            u = queue.popleft()
            comp.append(u)

            for v in adj[u]:
                if not visited[v]:
                    visited[v] = True
                    queue.append(v)

        components.append(comp)

    return components

def cluster_statistics(positions, L, R_cluster):
    """
    Restituisce alcune statistiche semplici sui cluster geometrici.
    """
    adj = adjacency_list_geometric(positions, L, R_cluster)
    comps = connected_components(adj)

```

```

sizes = [len(c) for c in comps]
sizes.sort(reverse=True)

return {
    "num_clusters": len(sizes),
    "largest_cluster": sizes[0] if sizes else 0,
    "cluster_sizes": sizes
}

```

A.10 Simulazione di una traiettoria

```

def run_vicsek(
    N,
    L,
    R,
    v0,
    eta,
    T,
    dt=1.0,
    seed=None,
    measure_clusters=False
):
    """
    Simula una traiettoria del modello di Vicsek per T passi.

    Restituisce un dizionario con:
    - positions_history
    - angles_history
    - order_history
    - cluster_history (opzionale)
    """
    rng = make_rng(seed)

    positions, angles = initialise_vicsek_state(N, L, rng=rng)

    positions_history = [positions[:]]
    angles_history = [angles[:]]
    order_history = [order_parameter(angles)]
    cluster_history = []

    if measure_clusters:
        cluster_history.append(cluster_statistics(positions, L, R))

    for _ in range(T):
        positions, angles = vicsek_step(
            positions=positions,

```

```

        angles=angles,
        L=L,
        R=R,
        v0=v0,
        eta=eta,
        dt=dt,
        rng=rng
    )

    positions_history.append(positions[:])
    angles_history.append(angles[:])
    order_history.append(order_parameter(angles))

    if measure_clusters:
        cluster_history.append(cluster_statistics(positions, L, R))

    return {
        "positions_history": positions_history,
        "angles_history": angles_history,
        "order_history": order_history,
        "cluster_history": cluster_history
    }

```

A.11 Tempo di formazione dell'ordine

```

def ordering_time(order_history, phi_threshold=0.8):
    """
    Restituisce il primo tempo t in cui  $\Phi(t)$  supera la soglia.
    Se non accade, restituisce None.
    """
    for t, phi in enumerate(order_history):
        if phi >= phi_threshold:
            return t
    return None

```

A.12 Medie temporali e fluttuazioni

```

def time_average(values, start_index=0):
    """
    Media temporale di una lista, a partire da un indice iniziale.
    """
    tail = values[start_index:]
    if len(tail) == 0:
        return None
    return sum(tail) / len(tail)

```

```

def susceptibility(order_history, start_index=0):
    """
    Stima empirica della quantita'
     $\chi = N * ( \langle \Phi^2 \rangle - \langle \Phi \rangle^2 )$ 
    Qui restituiamo solo la parte tra parentesi;
    il fattore N puo' essere moltiplicato a parte.
    """
    tail = order_history[start_index:]
    if len(tail) == 0:
        return None

    mean_phi = sum(tail) / len(tail)
    mean_phi2 = sum(phi * phi for phi in tail) / len(tail)

    return mean_phi2 - mean_phi * mean_phi

```

A.13 Esperimento Monte Carlo per studiare il ruolo del rumore

```

def noise_sweep(
    eta_values,
    N,
    L,
    R,
    v0,
    T,
    dt=1.0,
    transient_fraction=0.5,
    repetitions=10,
    base_seed=12345
):
    """
    Esegue un insieme di simulazioni per diversi valori di eta
    e restituisce statistiche aggregate del parametro d'ordine.

    Per ogni eta:
    - esegue 'repetitions' traiettorie indipendenti;
    - scarta una frazione iniziale come transiente;
    - calcola media e fluttuazioni di Phi.
    """
    results = []

    for k, eta in enumerate(eta_values):
        phi_means = []
        chi_values = []

```

```

for rep in range(repetitions):
    seed = base_seed + 1000 * k + rep

    sim = run_vicsek(
        N=N,
        L=L,
        R=R,
        v0=v0,
        eta=eta,
        T=T,
        dt=dt,
        seed=seed,
        measure_clusters=False
    )

    order_hist = sim["order_history"]
    start_index = int(transient_fraction * len(order_hist))

    phi_bar = time_average(order_hist, start_index=start_index)
    chi = susceptibility(order_hist, start_index=start_index)

    phi_means.append(phi_bar)
    chi_values.append(chi)

    mean_phi = sum(phi_means) / len(phi_means)
    mean_chi = sum(chi_values) / len(chi_values)

    results.append({
        "eta": eta,
        "mean_phi": mean_phi,
        "mean_chi_times_N": N * mean_chi
    })

return results

```

A.14 Esempio minimo di utilizzo

```

if __name__ == "__main__":
    N = 200
    L = 20.0
    R = 1.0
    v0 = 0.03
    eta = 0.4
    T = 500
    dt = 1.0

```

```

sim = run_vicsek(
    N=N,
    L=L,
    R=R,
    v0=v0,
    eta=eta,
    T=T,
    dt=dt,
    seed=42,
    measure_clusters=True
)

order_hist = sim["order_history"]
print("Phi iniziale =", order_hist[0])
print("Phi finale   =", order_hist[-1])

tau = ordering_time(order_hist, phi_threshold=0.8)
print("Tempo di ordinamento =", tau)

last_clusters = sim["cluster_history"][-1]
print("Numero di cluster finali =", last_clusters["num_clusters"])
print("Taglia del cluster piu' grande =", last_clusters["largest_cluster"])

```

A.15 Esempio per il diagramma ordine–rumore

```

if __name__ == "__main__":
    eta_values = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]

    results = noise_sweep(
        eta_values=eta_values,
        N=300,
        L=20.0,
        R=1.0,
        v0=0.03,
        T=800,
        dt=1.0,
        transient_fraction=0.5,
        repetitions=8,
        base_seed=2024
    )

    for row in results:
        print(
            "eta =",
            row["eta"],

```



```

    " mean_phi =",
    round(row["mean_phi"], 4),
    " N*chi =",
    round(row["mean_chi_times_N"], 4)
)

```

A.16 Possibili esercizi computazionali immediati

1. Simulare una traiettoria del modello per diversi valori di **eta** e confrontare qualitativamente gli snapshot finali.
2. Calcolare il parametro d'ordine medio $\bar{\Phi}$ in funzione del rumore.
3. Stimare il tempo di formazione dell'ordine per diversi valori della densita' $\rho = N/L^2$.
4. Misurare come cambia la taglia del cluster piu' grande al variare di **eta**.
5. Confrontare due regimi con la stessa densita' ma diversa combinazione di N e L.
6. Sostituire il vicinato metrico con un vicinato topologico basato sui k vicini piu' prossimi.

A.17 Nota metodologica

Il codice proposto e' volutamente semplice e privilegia la leggibilita'. In una versione piu' avanzata si potrebbero introdurre:

- **numpy** per velocizzare il calcolo vettoriale;
- strutture a celle spaziali per ridurre il costo della ricerca dei vicini;
- visualizzazioni in tempo reale delle particelle e dei vettori velocita';
- stime piu' raffinate delle correlazioni spaziali;
- studio della dipendenza dalla taglia del sistema.

Dal punto di vista didattico, questa implementazione ha pero' un vantaggio importante: rende completamente trasparente il passaggio da una regola microscopica locale alla formazione di una osservabile macroscopica come $\Phi(t)$.